GEM 520 Lab 6 - Supervised Image Classification

Oluwaseyi Ezekiel

01 November 2025

Contents

PART 1 - Supervised Classification . . . 1
PART 2 - Defining areas of training data . . . 1
PART 3 - Image classification and accuracy assessment . . . 2
Code answer . . . 13
Question 1 . . . 14
Question 2 . . . 14
Question 3 . . . 14
Question 4 . . . 15
Question 5 . . . 15
Part 4 - Classification Examination . . . 15

The R code to analyze the polygons that you have delineated, split the data into training and validation sets, classify the image with the Maximum Likelihood algorithm and generate the confusion matrix is provided. You just need to set the file path to your `classification_polygons_YourInitials.shp` at the beginning of PART 3.

However, you will have to type your own code here to calculate the overall accuracy, user accuracy and producer accuracy from the confusion matrix. **Note that the overall accuracy must be more than 75% to receive full marks**.

Make also sure to answer Q1, Q2, Q3, Q4 and Q5 in your R Markdown report.

This document reads in the Landsat imagery and the delineated polygons with relative file paths from the root directory of the lab folder, where the Rmd file is located. Make sure that `classification_polygons_YourInitials` files are stored in the `outputs` folder and that your Rmd file is located at the root of the directory

PART 1 - Supervised Classification

See PDF

PART 2 - Defining areas of training data

See PDF

PART 3 - Image classification and accuracy assessment

SET THE FILE NAME OF YOUR `classification_polygons_YourInital.shp` file in the following code chunk

```
my_polygons <- "data/classification_polygons_GEO.shp"
```

The following packages need to be installed on your machine. *DO NOT install the packages in the R Markdown document. Run the install.packages() commands in the console, in a fresh and clean R Studio session*

```
install.packages('terra', repos='https://rspatial.r-universe.dev')
install.packages("sf")
install.packages("RStoolbox")
install.packages("tidyverse")
install.packages("caret")
```

Once installed, we attach the packages

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.5.1

## Warning: package 'ggplot2' was built under R version 4.5.1

## Warning: package 'tibble' was built under R version 4.5.1

## Warning: package 'tidyr' was built under R version 4.5.1

## Warning: package 'readr' was built under R version 4.5.1

## Warning: package 'purrr' was built under R version 4.5.1

## Warning: package 'dplyr' was built under R version 4.5.1

## Warning: package 'stringr' was built under R version 4.5.1

## Warning: package 'forcats' was built under R version 4.5.1

## Warning: package 'lubridate' was built under R version 4.5.1
```

```
library(terra)
```

```
## Warning: package 'terra' was built under R version 4.5.1
```

```
library(sf)
```

```
## Warning: package 'sf' was built under R version 4.5.1
```

```r
library(RStoolbox)
```

```
## Warning: package 'RStoolbox' was built under R version 4.5.1
```

We start by reading the Landsat image into R

```r
ls_image <- rast("data/LC09_L2SP_047026_20240716_20240717_02_T1_SR_BSTACK.tif")

ls_image
```

```
## class       : SpatRaster
## size        : 1407, 1194, 6  (nrow, ncol, nlyr)
## resolution  : 30, 30  (x, y)
## extent      : 466215, 502035, 5371545, 5413755  (xmin, xmax, ymin, ymax)
## coord. ref. : WGS 84 / UTM zone 10N (EPSG:32610)
## source      : LC09_L2SP_047026_20240716_20240717_02_T1_SR_BSTACK.tif
## names       : blue, green, red, nir, swir1, swir2
## min values  :    0,     0,   0,   0,     0,     0
## max values  :    1,     1,   1,   1,     1,     1
```

```r
terra::plotRGB(ls_image, r = 3, g = 2, b = 1, stretch = "lin")
```

Then, we load the delineated polygons

```r
class_poly <- st_read(my_polygons)
```

```
## Reading layer ‘classification_polygons_GEO’ from data source
##   ‘C:\GEM520\Labs\Lab6\GEM520_Lab6_classification\data\classification_polygons_GEO.shp’
##   using driver ‘ESRI Shapefile’
## Simple feature collection with 107 features and 1 field
## Geometry type: POLYGON
## Dimension:     XY
## Bounding box:  xmin: 467113 ymin: 5371663 xmax: 501709.5 ymax: 5413337
## Projected CRS: WGS 84 / UTM zone 10N
```

```r
# Make sure that the geometry is valid
class_poly <- st_make_valid(class_poly)

# Tranform lc_class to factor
class_poly <- class_poly %>%
  mutate(lc_class = factor(lc_class,
                           levels = c("Broadleaf Forest", "Coniferous Forest", "Exposed soil and rocks"

# Plot
terra::plotRGB(ls_image, r = 3, g = 2, b = 1, stretch = "lin")
plot(class_poly[, "lc_class"], add = TRUE)
```

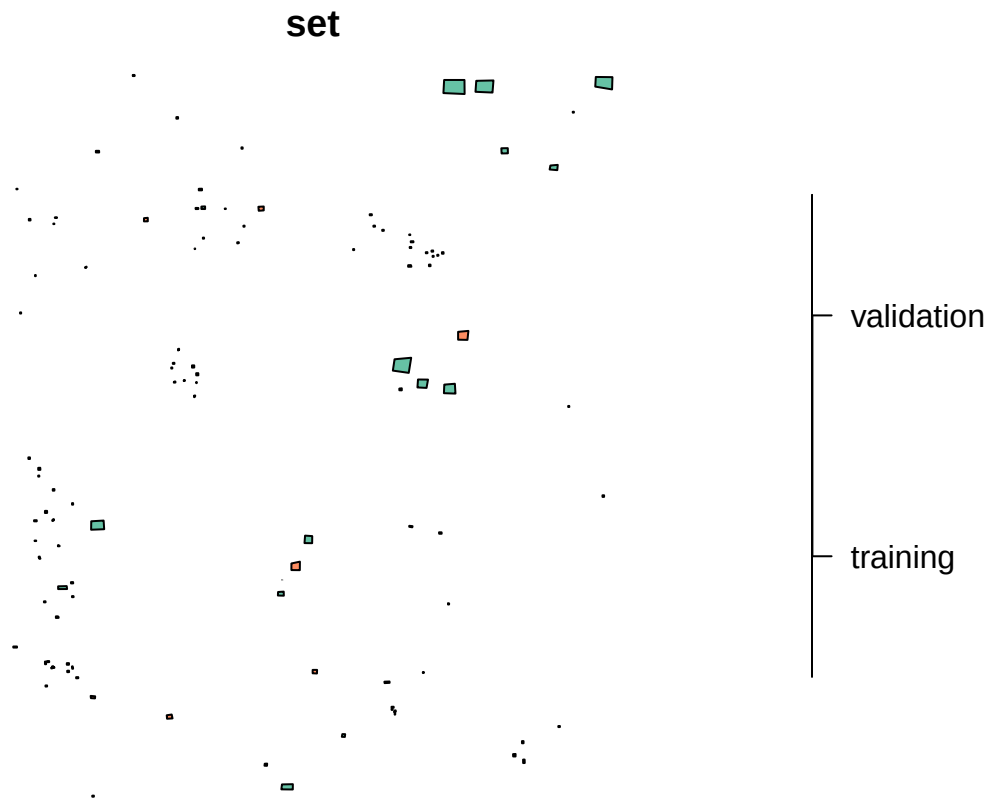
Here is a summary of the number of polygons per class

```r
poly_summary <- class_poly %>%
  st_drop_geometry() %>%
  group_by(lc_class) %>%
  summarize(n_poly = n())

poly_summary
```

```
## # A tibble: 7 x 2
##   lc_class                n_poly
##   <fct>                    <int>
## 1 Broadleaf Forest            16
## 2 Coniferous Forest           24
## 3 Exposed soil and rocks      13
## 4 High density developed      10
## 5 Low density developed       10
## 6 Non-forest vegetation       11
## 7 Water                       23
```

For each land cover class will use 70% of the polygons to train the classification algorithm and the remaining 30% for validation.

```r
# Assign a unique ID to each polygon
class_poly <- tibble::rowid_to_column(class_poly, var = "ID")

set.seed(1234)

# Sample 70% of the polygons in each land cover class
poly_train <- class_poly %>%
  group_by(lc_class) %>%
  sample_frac(0.7) %>%
  mutate(set = "training") %>% st_cast(to = 'POLYGON')

# Use the ID field to select the polygons for validation
poly_val <- class_poly %>%
  filter(!ID %in% poly_train$ID) %>%
  mutate(set = "validation") %>% st_cast(to = 'POLYGON')

poly_set <- rbind(poly_train,
                  poly_val)

# Plot poly_set
plot(poly_set[, "set"])
```



We now extract the values of the Landsat image pixels in the polygons

```
# Add join column to our polygon set and remove the old one
poly_set <- poly_set %>% tibble::rowid_to_column(var = "join_id") %>% select(-ID)

# Extract pixel values for each polygon
poly_set_vals <- terra::extract(ls_image, vect(poly_set))

# Join with LC class via an inner_join (where new join_id == ID from terra)
poly_set_vals <- inner_join(poly_set, poly_set_vals, join_by(join_id == ID)) %>%
  st_drop_geometry()
```

Each row of `poly_set_vals` corresponds to one pixel of the Landsat image.

```
poly_set_vals
```

```
## # A tibble: 6,751 x 9
## # Groups:   lc_class [7]
##    join_id lc_class         set        blue  green    red   nir swir1  swir2
##  *   <dbl> <fct>            <chr>     <dbl>  <dbl>  <dbl> <dbl> <dbl>  <dbl>
##  1       1 Broadleaf Forest training 0.0168 0.0341 0.0185 0.355 0.117 0.0424
##  2       1 Broadleaf Forest training 0.0169 0.0374 0.0188 0.430 0.139 0.0480
##  3       1 Broadleaf Forest training 0.0174 0.0406 0.0235 0.421 0.146 0.0549
##  4       1 Broadleaf Forest training 0.0177 0.0349 0.0186 0.364 0.126 0.0467
##  5       1 Broadleaf Forest training 0.0197 0.0389 0.0195 0.440 0.150 0.0520
```

```
##  6       1 Broadleaf Forest training 0.0176 0.0383 0.0190 0.415 0.144 0.0533
##  7       1 Broadleaf Forest training 0.0212 0.0366 0.0215 0.329 0.124 0.0481
##  8       1 Broadleaf Forest training 0.0227 0.0394 0.0209 0.410 0.147 0.0526
##  9       1 Broadleaf Forest training 0.0195 0.0369 0.0193 0.386 0.131 0.0483
## 10       2 Broadleaf Forest training 0.0261 0.0455 0.0260 0.444 0.162 0.0645
## # i 6,741 more rows
```

We can check the number of pixels per class and training / validation set

```
poly_stats <- poly_set_vals %>%
  group_by(set, lc_class) %>%
  summarize(n_px = n())
```

```
## `summarise()` has grouped output by 'set'. You can override using the `.groups`
## argument.
```

```
poly_stats
```

```
## # A tibble: 14 x 3
## # Groups:   set [2]
##    set        lc_class                n_px
##    <chr>      <fct>                  <int>
##  1 training   Broadleaf Forest          73
##  2 training   Coniferous Forest         90
##  3 training   Exposed soil and rocks   192
##  4 training   High density developed    66
##  5 training   Low density developed     46
##  6 training   Non-forest vegetation     80
##  7 training   Water                   5132
##  8 validation Broadleaf Forest          41
##  9 validation Coniferous Forest         37
## 10 validation Exposed soil and rocks    83
## 11 validation High density developed    30
## 12 validation Low density developed     20
## 13 validation Non-forest vegetation     42
## 14 validation Water                    819
```

We can pivot the data from a wide to long format

```
poly_set_vals_long <- pivot_longer(poly_set_vals,
                                   blue:swir2,
                                   names_to = "band",
                                   values_to = "reflectance")

poly_set_vals_long
```

```
## # A tibble: 40,506 x 5
## # Groups:   lc_class [7]
##    join_id lc_class         set      band  reflectance
##      <dbl> <fct>            <chr>    <chr>       <dbl>
##  1       1 Broadleaf Forest training blue       0.0168
```

```
## 2       1 Broadleaf Forest training green      0.0341
## 3       1 Broadleaf Forest training red        0.0185
## 4       1 Broadleaf Forest training nir        0.355
## 5       1 Broadleaf Forest training swir1      0.117
## 6       1 Broadleaf Forest training swir2      0.0424
## 7       1 Broadleaf Forest training blue       0.0169
## 8       1 Broadleaf Forest training green      0.0374
## 9       1 Broadleaf Forest training red        0.0188
## 10      1 Broadleaf Forest training nir        0.430
## # i 40,496 more rows
```

And calculate some summary statistics for each band and land cover class: mean, 5th quantile and 95th quantile of reflectance.

```
spectral_sign <- poly_set_vals_long %>%
  group_by(lc_class, band) %>%
  summarize(r_mean = mean(reflectance, na.rm = TRUE),
            r_q05 = quantile(reflectance, 0.05, na.rm = TRUE),
            r_q95 = quantile(reflectance, 0.95, na.rm = TRUE))
```

```
## `summarise()` has grouped output by 'lc_class'. You can override using the
## `.groups` argument.
```

```
spectral_sign
```

```
## # A tibble: 42 x 5
## # Groups:   lc_class [7]
##    lc_class          band  r_mean   r_q05  r_q95
##    <fct>             <chr>  <dbl>   <dbl>  <dbl>
##  1 Broadleaf Forest  blue  0.0185 0.0145  0.0257
##  2 Broadleaf Forest  green 0.0394 0.0347  0.0474
##  3 Broadleaf Forest  nir   0.361  0.265   0.440
##  4 Broadleaf Forest  red   0.0235 0.0174  0.0350
##  5 Broadleaf Forest  swir1 0.134  0.102   0.171
##  6 Broadleaf Forest  swir2 0.0523 0.0387  0.0748
##  7 Coniferous Forest blue  0.0158 0.00867 0.0263
##  8 Coniferous Forest green 0.0306 0.0235  0.0417
##  9 Coniferous Forest nir   0.256  0.206   0.320
## 10 Coniferous Forest red   0.0185 0.0130  0.0273
## # i 32 more rows
```

We can now visualize the spectral signature of each land cover class

```
# Wavelength corresponding to each band
bands_wavelength <- read_csv("data/bands_wavelength.csv")
```

```
## Rows: 6 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): band
## dbl (1): wavelength
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

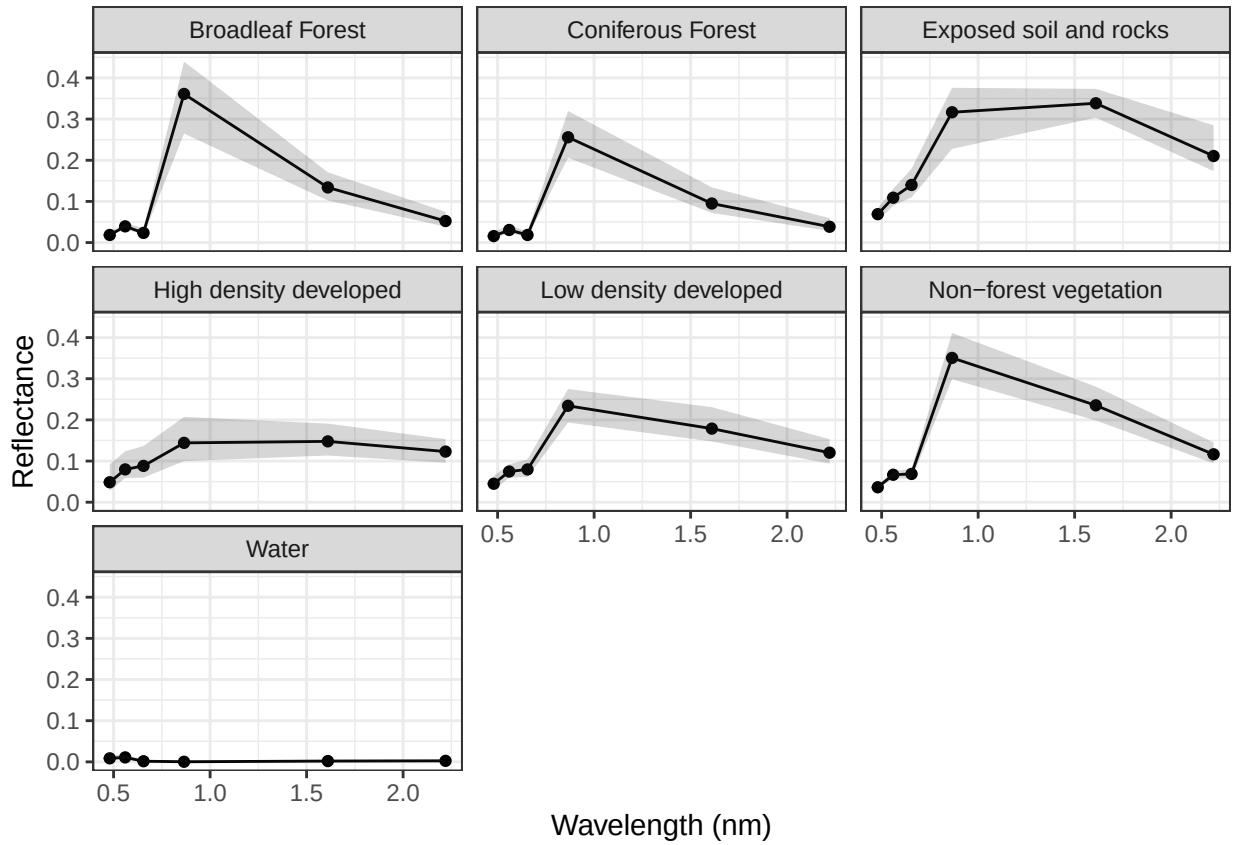
```r
bands_wavelength
```

```
## # A tibble: 6 x 2
##   band  wavelength
##   <chr>      <dbl>
## 1 blue       0.48
## 2 green      0.56
## 3 red        0.655
## 4 nir        0.865
## 5 swir1      1.61
## 6 swir2      2.22
```

```r
# Join wavelength
spectral_sign <- inner_join(spectral_sign, bands_wavelength)
```

```
## Joining with `by = join_by(band)`
```

```r
# Graph
ggplot(spectral_sign, aes(x = wavelength, y = r_mean, group = 1)) +
  geom_point() +
  geom_line() +
  geom_ribbon(aes(ymin = r_q05, ymax = r_q95), alpha = 0.2) +
  facet_wrap(vars(lc_class)) +
  theme_bw() +
  labs(x = "Wavelength (nm)",
       y = "Reflectance")
```



We can now use the function `superClass()` from the `RSToolbox` package to perform the classification and accuracy assessment. The argument `model = "mlc"` is used to select the Maximum Likelihood algorithm for classification. We provide the polygons used for training and validation under the arguments `trainData` and `valData`. The function will sample `500` pixels from the training polygons (argument `nSamples = 500`) per land cover class and use this sample to train the classification algorithm. Similarly, validation will be performed on a sample of `500` pixels of the validation polygons per land cover class. Note that for some classes there might not be enough pixels to reach `500` observations. In that case, training / validation would be performed on < `500` pixels per land cover class. It is important to try to balance the number of observation per classes before training the classification model and assessing its accuracy. Otherwise, the overall accuracy derived from the confusion matrix could be biased towards the classes with the largest number of observations.

For example, it is easier to draw large polygons across water (and hence getting a lot of training/validation data) than it is over broadleaf forest. Let's assume that we use 1000 pixels to assess the accuracy of water and 20 pixels to assess the accuracy of broadleaf vegetation. For water, 900 out of 1000 pixels (90%) were classified correctly whereas only 5 out of 20 pixels were classified correctly for broadleaf.

The overall accuracy would be:

```
OA_unbalanced <- (900 + 5) / (1000 + 20)

OA_unbalanced
```

```
## [1] 0.8872549
```

Now let's assume that 20 pixels were used for both water and broadleaf to assess the classification accuracy. For water, 18 out of 20 pixels (still 90%) were classified correctly and 5 out of 20 pixels were classified correctly for broadleaf.

The overall accuracy would be:

```
OA_balanced <- (18 + 5) / (20 + 20)

OA_balanced
```

```
## [1] 0.575
```

In the first case where classes are unbalanced we obtain an overall accuracy of 89%. Because there are much more pixels in water than in broafleaf in the first case, the overall accuracy is very high and doesn't reflect the fact that the broadleaf class is poorly classified. In the second case, where classes are balanced, we obtain an overall accuracy of 58%. This illustrates that the classification is not as good as we might think with the first case.

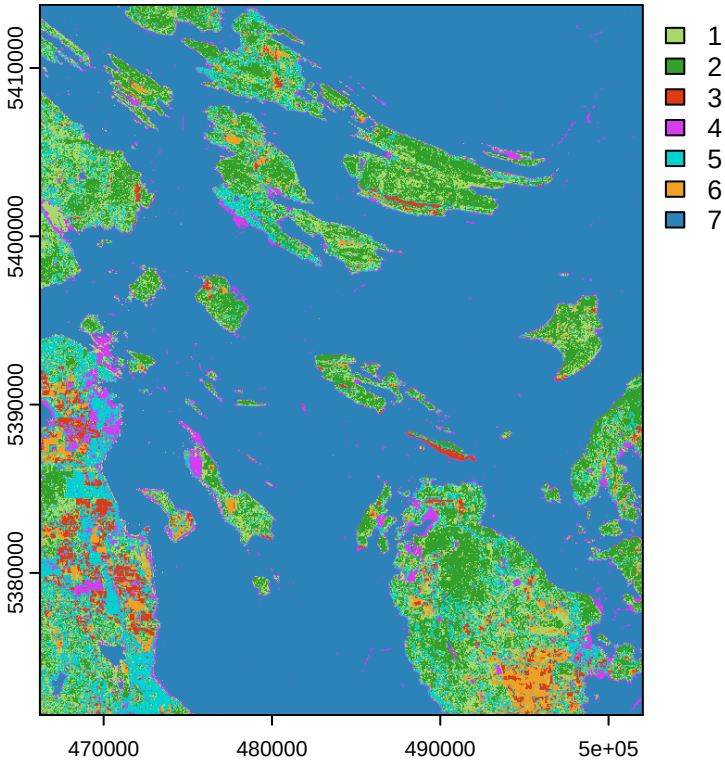
The superClass() function might take a few minutes to run

```
set.seed(1234)

poly_train <- poly_train %>% rename(class = lc_class)
poly_val <- poly_val %>% rename(class = lc_class)

mlc_model <- superClass(img = ls_image,
                        trainData = poly_train,
                        valData = poly_val,
                        responseCol = "class",
                        model = "mlc",
                        nSamples = 500)
```

```
## Warning: package 'caret' was built under R version 4.5.1

## Loading required package: lattice

##
## Attaching package: 'caret'

## The following object is masked from 'package:purrr':
##
##     lift
```

The `superClass()` function returns a list with multiple objects. The classified map is stored in the element of the list called `map`. The trained model is stored in the element `model`. The predictions of the model at the validation data are stored in a data.frame called `validationSamples` located in the element of `mlc_model` called `validation`. The column `reference` is the land cover class that you have assigned and the column `prediction` is the land cover class predicted by the model.

```
classified_map <- mlc_model$map

# Write the classified map as a tif file
writeRaster(classified_map,
            filename = "outputs/classified_map.tif",
            overwrite = TRUE)

# Plot with colors
plot(classified_map,
     col = c('#A6D96A','#33A02C','#DE3B13','#D63CF1','#00D2D2','#F1A026','#2B83BA'))
```




```r
# Validation df
val_preds <- mlc_model$validation$validationSamples

head(val_preds)
```

```
## Simple feature collection with 6 features and 3 fields
## Geometry type: POINT
## Dimension:     XY
## Bounding box:  xmin: 484470 ymin: 5378820 xmax: 499470 ymax: 5411190
## Projected CRS: WGS 84 / UTM zone 10N
##   reference prediction    cell               geometry
## 1     Water      Water  102598 POINT (499440 5411190)
## 2     Water      Water  102599 POINT (499470 5411190)
## 3     Water      Water 1390426 POINT (484500 5378820)
## 4     Water      Water 1386843 POINT (484470 5378910)
## 5     Water      Water 1384458 POINT (484560 5378970)
## 6     Water      Water 1389233 POINT (484530 5378850)
```

The confusion matrix can be created from `val_preds` using the `table()` function (base R package). The columns represent the reference classes (classes you have assigned) and the rows represent the predictions of the model.

```r
conf_matrix <- table(st_drop_geometry(val_preds[, c("prediction", "reference")]))

knitr::kable(conf_matrix)
```

	Broadleaf Forest	Coniferous Forest	Exposed soil and rocks	High density developed	Low density developed	Non-forest vegetation	Water
Broadleaf Forest	39	0	0	0	0	0	0
Coniferous Forest	0	33	0	0	0	0	0
Exposed soil and rocks	0	0	45	0	0	0	0
High density developed	0	0	9	29	0	0	0
Low density developed	0	0	0	1	12	1	0
Non-forest vegetation	0	0	0	0	0	41	0
Water	0	0	0	0	0	0	683

The `table()` function returns a `matrix` object with the predicted classes in the rows and the reference classes in the columns. The diagonal of a matrix can be returned as a `vector` using the `diag()` function. The row and column sums can be returned with the `rowSums()` and `colSums()` functions. The sum of all elements of a matrix can be obtained with the `sum()` function.

Use the functions described above to calculate the overall accuracy (`OA`), producer accuracy (`PA`) and user accuracy (`UA`) of your classification.

Note that the overall accuracy must be more than 75% to receive full marks

Code answer

```r
# Assuming you have a confusion matrix
#conf_matrix <- table(predicted, reference)

# Overall Accuracy
OA <- sum(diag(conf_matrix)) / sum(conf_matrix)

# Producer Accuracy
PA <- diag(conf_matrix) / colSums(conf_matrix)

# User Accuracy
UA <- diag(conf_matrix) / rowSums(conf_matrix)

# Print results
print(paste("Overall Accuracy:", round(OA, 3)))
```

```
## [1] "Overall Accuracy: 0.988"
```

```r
print("Producer Accuracy:")
```

```
## [1] "Producer Accuracy:"
```

```r
print(round(PA, 3))
```

```
##       Broadleaf Forest      Coniferous Forest Exposed soil and rocks
##                  1.000                  1.000                  0.833
## High density developed  Low density developed  Non-forest vegetation
##                  0.967                  1.000                  0.976
##                  Water
##                  1.000
```

```r
print("User Accuracy:")
```

```
## [1] "User Accuracy:"
```

```r
print(round(UA, 3))
```

```
##       Broadleaf Forest      Coniferous Forest Exposed soil and rocks
##                  1.000                  1.000                  1.000
## High density developed  Low density developed  Non-forest vegetation
##                  0.763                  0.857                  1.000
##                  Water
##                  1.000
```

Answer the following questions in brief answers (1-5 lines each)

Question 1

What is a supervised classification process and how does it differs from unsupervised classification? A supervised classification is one that requires humans to provide training data to guide the computer in the classification. Distinct pixels are digitized for each defined unique classes and are used by the classifier algorithm to group similar pixels. Unlike supervised classification, no training data are provided in unsupervised classification. Instead, the computer uss clustering algorithms to automatically group pixels based on their reflectance patterns, after which interpretation of the clusters would be done by a human.

Question 2

Why do we need distinct training and validation data, and why is important they do not overlap? It is important to separate training from validation data to ensure fairness in our accuracy results. If our classifier is tested using the same pixels used in training it, the classifier would literally be testing itself on data it has already seen. Although this would increase the accuracy (in fact ~100%), but it does not guarantee the reliability of the classifier.

Question 3

How does Maximum Likelihood classification work? Maximum Likelihood Classification is a supervised classification method that calculates the probability that each pixel belongs to a particular class, using the mean and variance of the training data. It assumes that the data follow a normal distribution and assigns each pixel to the class with the highest probability.

Question 4

Which classes achieved the worst producer and user accuracies, why do you think this is? Exposed soil and rocks had the lowest producer accuracy (83.3%) while 'High density developed' had the lowest user accuracy (76.3%). I think this happened because of the possible overlap in the pixels I used to represent both land cover classes. Features like roads, concrete which may have been captured as 'exposed soil and rocks' might have easily be confused with built-up surfaces in high density developed areas.

Question 5

Which classes achieved the best results, why do you think this is? Water, Broadleaf Forest, and Coniferous Forest had the best accuracies (100%). First, I think this is because they have very distinct spectral signatures in the NIR region. SO, it was easy for me to identify and separate the classes using the standard falso color composites. I also think that the relative abundance of pixels in these classes allowed the creation of more polygons, and this could have improved the accuracy.

Part 4 - Classification Examination

Now, load the classified map `outputs/classified_map.tif` back into QGIS. Navigate around the classified image and compare it to the original image. Ask yourself: was this a successful classification? Does the distribution of classes make sense? Are there any areas of major misclassification? Would you feel comfortable and confident passing along this map to someone who would then make "real world" decisions based off of it? In order to answer these questions (in your head. . . these are not actual graded questions), feel free to review the Google Satellite imagery with this classified map.

Overall, this was a successful classification, with an overall accuracy of about 98.8% and relatively high user and producer accuracies for each land cover class. Despite minor confusion among some of the classes, I am confident passing this map to someone who would make real decisions based on it.